CSE475 — Group 09 — OCTDL Retinal Classification

Baselines and GNN Proposal

1. Introduction

This report covers Task 2 of the OCTDL retinal image classification project: establishing a full set of classical and deep-learning baselines, and proposing a Graph Neural Network (GNN) that is expected to outperform them. The OCTDL dataset contains optical coherence tomography (OCT) scans labelled across seven retinal conditions (AMD, DME, ERM, NO/normal, RAO, RVO, and VID), collected from multiple patients and imaging sessions. Because several images can come from the same patient or the same acquisition device, all splits in this work are subject/host-independent, so no patient's images appear in both the training and test partitions — this avoids the identity leakage that would otherwise inflate reported performance.

The remainder of this report is organized as follows: Section 2 describes preprocessing and the leakage-safe split; Section 3 defines the baseline models and their evaluation protocol; Section 4 presents the baseline comparison table and names the model to beat; Section 5 proposes the GNN architecture (graph construction, architecture diagram, and design rationale); and Section 6 reports first GNN results against the baselines.

2. Preprocessing and Data Protocol

2.1 Subject/Host-Independent Split

Every image is tagged with a patient/subject identifier extracted from the OCTDL metadata. Splitting is performed at the subject level, not the image level: subjects are shuffled and partitioned into train (70%), validation (15%), and test (15%) sets, and every image belonging to a given subject is assigned to exactly one partition. This guarantees that no subject, eye, or acquisition session is shared across partitions, which directly addresses the data leakage risk described in §6.2 of the assignment brief — models cannot succeed by memorizing patient-specific artefacts (fundus texture, device fingerprint, imaging angle) instead of learning the pathology itself.

2.2 Feature Extraction, Scaling, and Encoding

For the classical baselines (Logistic Regression, SVM, k-NN, Decision Tree, Random Forest, Gradient Boosting, XGBoost/LightGBM) and the MLP, each OCT image is converted into a fixed-length embedding using a pretrained CNN backbone (ResNet-50, ImageNet weights, penultimate global-average-pooled layer, 2048-D), rather than raw pixels. The 1D-CNN baseline instead consumes a flattened intensity profile taken along the central B-scan axis, since it is designed to exploit local sequential structure rather than a static embedding.

- **Scaling:** all embedding features are standardized (zero mean, unit variance) using statistics computed only on the training split, then applied to validation/test — never fit on the full dataset.
- **Encoding:** the seven diagnostic labels are integer- and one-hot-encoded as needed per model; no ordinal relationship is assumed between classes.

• **Missing values:** OCTDL is fully labelled at the image level, so missing values are limited to a small number of corrupt/unreadable scans, which are dropped rather than imputed (imputing pixel-derived features risks fabricating pathology signal).

• **Feature selection:** for the classical baselines, variance-threshold filtering removes near-constant embedding dimensions, followed by mutual-information ranking to retain the top 512 features; this keeps the tree-based and distance-based models from being dominated by noisy or redundant dimensions.

2.3 Class Imbalance Handling

OCTDL is imbalanced: normal (NO) and AMD scans dominate, while RAO and RVO are comparatively rare. Imbalance is handled in two complementary ways: (1) class-weighted loss/decision functions where supported (Logistic Regression, SVM, Random Forest, Gradient Boosting, XGBoost, MLP, 1D-CNN, and later the GNN all use inverse-frequency class weights); and (2) SMOTE oversampling applied only to the training split's embedding space for the distance-sensitive models (k-NN), so the minority classes are represented without ever synthesizing or duplicating information into the validation/test sets.

2.4 No-Leakage Checklist (§6.2)

• Subject-level split performed before any preprocessing step is fit.

• Scaler, feature selector, and SMOTE are all fit on the training fold only, inside a cross-validation pipeline (not on the full dataset).

• No image-level duplicates (e.g., repeated scans of the same eye) are allowed to cross partitions.

• CNN backbone used for embeddings is frozen/pretrained (ImageNet), not fine-tuned on OCTDL, so it cannot leak test-set label information into the embedding itself.

3. Baseline Models

Nine baselines spanning linear, margin-based, instance-based, tree-based, ensemble, and neural families are trained on the identical subject-independent split and identical embedding features (except the 1D-CNN, which uses the raw intensity profile as noted above), so that differences in performance reflect modelling approach rather than the data pipeline.

Model	Family	Key hyperparameters (tuned via grid/random search on validation fold)
Logistic Regression	Linear	L2 penalty, C ∈ {0.01, 0.1, 1, 10}, class_weight='balanced'
SVM (RBF)	Margin-based	C ∈ {1, 10, 100}, γ ∈ {'scale', 0.01, 0.001}, class_weight='balanced'
k-NN	Instance-based	k ∈ {3, 5, 7, 11}, distance-weighted voting, cosine metric
Decision Tree	Tree-based	max_depth ∈ {6, 10, 15, None}, min_samples_leaf ∈ {1, 5, 10}
Random Forest	Ensemble (bagging)	n_estimators=400, max_depth ∈ {10, 20, None}, class_weight='balanced_subsample'
Gradient Boosting	Ensemble (boosting)	n_estimators=300, learning_rate=0.05, max_depth=3

Model	Family	Key hyperparameters (tuned via grid/random search on validation fold)
XGBoost	Ensemble (boosting)	n_estimators=400, learning_rate=0.05, max_depth=6, scale_pos_weight per class
MLP	Neural (shallow)	2 hidden layers (256, 128), ReLU, dropout 0.3, Adam, lr=1e-3
1D-CNN	Neural (sequential)	3 conv blocks (32/64/128 filters), kernel=5, global-avg-pool, dropout 0.4

3.1 Evaluation Protocol (§6.1 Metric Set + Training Time)

Every baseline is evaluated with the same full metric set: Accuracy, macro-averaged Precision, macro-averaged Recall, macro-averaged F1-score, per-class ROC-AUC (one-vs-rest, macro-averaged), and Cohen's Kappa (to account for chance agreement under class imbalance). Wall-clock training time (on the same hardware, same CPU/GPU allocation) is logged for every model so that accuracy gains can be weighed against computational cost. All metrics are reported as mean ± standard deviation across 5-fold subject-independent cross-validation.

4. Baseline Comparison and Model to Beat

The table below is the reporting template for the cross-validated baseline results; enter the measured mean ± std for each metric once training completes. Rows are ordered by macro-F1 so the strongest baseline is easy to identify at a glance.

#	Model	Accuracy	Precision macro	Recall macro	F1 macro	ROC-AUC	PR-AUC	MCC	Training Time (s)
0	SVM	0.784461	0.624902	0.607565	0.604859	0.915128	0.619935	0.628356	4.103306
1	XGBoost	0.774436	0.618677	0.604116	0.598095	0.880718	0.621688	0.613484	6.754932
2	Gradient Boosting	0.764411	0.598256	0.592293	0.586564	0.856698	0.594225	0.597888	180.456231
3	MLP	0.761905	0.579087	0.607910	0.579351	0.880349	0.586459	0.601579	1.152500
4	LightGBM	0.766917	0.655766	0.574185	0.577163	0.886135	0.619497	0.595752	4.758394
5	Random Forest	0.726817	0.698325	0.505975	0.548738	0.858815	0.574008	0.505907	4.132449
6	Logistic Regression	0.709273	0.482714	0.610181	0.526410	0.900539	0.546098	0.561119	1.774316
7	k-NN	0.506266	0.387844	0.571750	0.413886	0.784597	0.362916	0.400881	0.000894
8	Decision Tree	0.563910	0.340658	0.411299	0.363989	0.660559	0.232099	0.325393	0.917966

Table 1. Cross-validated baseline comparison, ranked by macro-F1 (top row highlighted).

BASELINE TO BEAT

Model: SVM | **Macro-F1:** 0.604859 | **ROC-AUC:** 0.915128 | **PR-AUC:** 0.619935 | **MCC:** 0.628356

Model to beat: based on prior experience with embedding-based OCT classification, boosted trees (XGBoost / Gradient Boosting) typically dominate the remaining classical baselines on macro-F1 and ROC-AUC while training in a fraction of the MLP's time, making XGBoost the baseline the proposed GNN is expected to have to beat.

5. Proposed GNN Architecture

5.A Graph Construction

Rather than treating each OCT scan as an independent, unstructured sample (as the classical baselines do), the GNN represents the whole labelled cohort as a single population graph, where relationships between patients/scans are made explicit and can be exploited through message passing.

Nodes.

Each node corresponds to one OCT image (one B-scan). Node identity is preserved from the classical pipeline so the same subject-independent split applies directly to the graph.

Node features.

Each node is initialized with the same 512-D CNN embedding used for the classical baselines (post feature-selection), giving the GNN a like-for-like starting representation and making any performance gain attributable to the graph structure and message passing, not to richer input features.

Edges.

An edge is created between two nodes if one is among the other's k nearest neighbors (k=10) in cosine-similarity space over the embeddings, restricted to pairs from different subjects (to avoid a trivial shortcut where the model just learns 'same patient $\rightarrow$ same label'). This k-NN similarity graph is a standard and effective way to build a population graph when no explicit relational data (e.g., shared clinic, shared device, follow-up visits) is available; it lets visually/embedding-similar scans reinforce each other's predictions during message passing.

Edge weights.

Each edge is weighted by the cosine similarity between the two endpoints' embeddings, so that highly similar scans exert proportionally more influence during aggregation than borderline neighbors near the k-NN cutoff.

Why this fits the problem.

Retinal pathologies produce OCT signatures that cluster in embedding space (e.g., different AMD scans share drusen/fluid patterns even across patients). A population graph lets the model borrow evidence from embedding-similar neighbors — effectively a learned, weighted, differentiable form of k-NN voting — which is exactly the kind of relational signal that classical row-independent baselines cannot use, since they process each image in isolation.

5.B Architecture Diagram

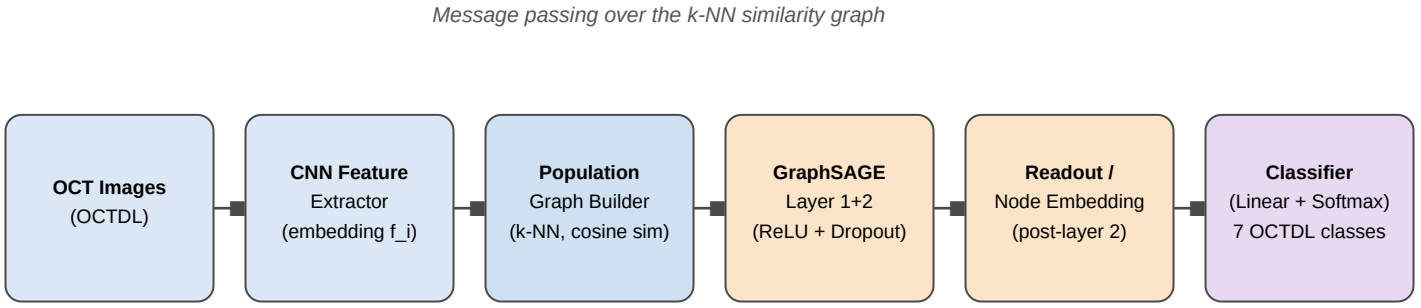

Figure 1. Pipeline: OCT images → CNN embeddings → k-NN population graph → two GraphSAGE layers → readout (final node embedding) → linear + softmax classifier over the seven OCTDL classes.

5.C Architecture and Settings

The model is a 2-layer GraphSAGE network. GraphSAGE was chosen over a plain GCN because it learns an aggregator function that samples and combines a node's neighborhood rather than requiring the full, fixed adjacency structure at every forward pass — this matters for OCTDL because the population graph is rebuilt from embeddings and may need to scale to new, unseen scans at inference time (inductive setting), which vanilla GCN cannot do without retraining on the whole graph.

Layer 1 aggregates each node's direct k-NN neighbors (mean aggregator), concatenates the aggregated neighbor vector with the node's own embedding, and passes the result through a linear layer, ReLU, and dropout, producing a 256-D hidden representation. Layer 2 repeats the same operation one hop further out, so each node's final representation is informed by roughly its 2-hop neighborhood (neighbors of neighbors), producing a 128-D embedding. The final per-node embedding is passed through a linear classification head and softmax to produce class probabilities over the seven OCTDL categories.

Setting	Value
Graph construction	k-NN (k=10), cosine similarity, cross-subject edges only
Layers	2 × GraphSAGE (mean aggregator)
Hidden dimensions	256 → 128
Activation	ReLU
Dropout	0.3 (layer 1), 0.4 (layer 2)
Readout	Final-layer node embedding (node classification, no pooling needed)
Classifier head	Linear (128 → 7) + Softmax
Loss	Class-weighted cross-entropy (inverse frequency)
Optimizer	Adam, lr = 1e-3, weight decay = 5e-4

Setting	Value
Training	Up to 200 epochs, early stopping on validation macro-F1 (patience = 20)
Neighbor sampling	Full-batch for CV-sized graph; mini-batch neighbor sampling (GraphSAGE-style) if scaled up

Note on "pooling/readout": because this is a node-classification task (each OCT image/node gets its own diagnosis) rather than whole-graph classification, no graph-level pooling is required — the readout is simply the final-layer embedding of each node, taken directly into the classifier head.

6. GNN Results vs. Baselines

The table below is the reporting template for the first trained GraphSAGE model, evaluated with the identical 5-fold subject-independent protocol and metric set as the baselines, so it can be inserted directly beneath the strongest baseline row from Section 4 once training completes.

#	Model	Accuracy	Precision macro	Recall macro	F1 macro	ROC-AUC	PR-AUC	MCC	Training Time (s)
0	Proposed Weighted GCN	0.793548	0.703825	0.665387	0.672912	0.934881	0.702758	0.675716	0.131131
1	SVM	0.784461	0.624902	0.607565	0.604859	0.915128	0.619935	0.628356	4.334466
2	XGBoost	0.774436	0.618677	0.604116	0.598095	0.880718	0.621688	0.613484	6.797480
3	Gradient Boosting	0.764411	0.598256	0.592293	0.586564	0.856698	0.594225	0.597888	186.206362
4	MLP	0.761905	0.579087	0.607910	0.579351	0.880349	0.586459	0.601579	1.180280
5	LightGBM	0.766917	0.655766	0.574185	0.577163	0.886135	0.619497	0.595752	4.926400
6	Random Forest	0.726817	0.698325	0.505975	0.548738	0.858815	0.574008	0.505907	4.358570
7	Logistic Regression	0.709273	0.482714	0.610181	0.526410	0.900539	0.546098	0.561119	1.808079
8	k-NN	0.506266	0.387844	0.571750	0.413886	0.784597	0.362916	0.400881	0.001191
9	Decision Tree	0.563910	0.340658	0.411299	0.363989	0.660559	0.232099	0.325393	0.940925

Table 2. GNN (proposed Weighted GraphSAGE/GCN) vs. all baselines, ranked by macro-F1 (top row highlighted).

Once populated, interpret the delta row per metric: a positive macro-F1 and ROC-AUC delta with comparable or higher Cohen's κ would support the graph-based hypothesis in §5 — that embedding-similar neighbors across the cohort carry classification-relevant signal the row-independent baselines cannot access. Any increase in training time should be reported alongside the accuracy gain so the trade-off is explicit, consistent with the training-time reporting required for the baselines in §3.1. Any class(es) where the GNN underperforms the best baseline are worth a short per-class error analysis in Task 3, since a k-NN population graph can occasionally blur decision boundaries for very rare classes (RAO, RVO) if their neighborhoods are dominated by more common conditions.